Question

Summarize 2026 business impact. Highlight key contributions, goal outcomes, and Leadership Principles in action. Where possible, include evidence such as metrics, measurable results, examples, or feedback.

Employee

Comment

2026 Impact Summary — Chi Zhang

Software Engineer 2, Pricing & Money Movement (Snowglobe) · Heading Towards Senior
Cycle: November 2025 – October 2026

Summary

Snowglobe is Braintree's Snowflake-based platform for calculating, journaling and reconciling merchant fees. Building on an Exceptional rating last cycle, this year I moved from delivering features on Snowglobe to owning how it stays correct and designing where it goes next:

- Correctness. I found and fixed a silent fee gap affecting ~196 merchants, and caught a 12.04M-transaction interchange gap.
- Recovery. I recovered refund fees for two of our largest Venmo merchants and reconciled them with their finance teams.
- Platform. I built the data-quality framework my teammates now build on.
- Next revenue lines. I designed and built two new ones; both are in final review, with launch gated on Product and Tax decisions.

- Fee gaps closed: Silent missing fees on standard-ACH payments (~196 merchants, ~$11M/day of payment volume affected): root-caused and structurally fixed · a 12.04M-transaction interchange gap caught by my check
- Money recovered: ~$171K of Temu refund fees across ~196K refunded credits, reconciled payout by payout with Temu's finance team · $33.9K for SHEIN, backfilled and paid out
- Platform reach: 22 quality checks on my framework (13 written by three other engineers), covering 15,913 merchant accounts
- On-call load: Failed runs on the noisiest check: 7.0 → 0.23 per week · pinless false-error noise eliminated (~31M junk rows in a July sample)
- New revenue lines: US Gross Settlement fee and AU refund charge (~AUD 100K a year): designed and built, in final review; revenue starts at launch, gated on Product and Tax decisions
- Output: 4 ADRs · 40 design, RCA and runbook pages · 40 merged PRs (36 in Snowglobe, 13th of 84 contributors) · 74 PRs reviewed

Ownership: I treat Snowglobe's correctness as mine

Silent fee gap on standard ACH. A health check I built flagged settled bank payments with no Braintree fee: ~196 merchants and ~$11M/day of volume, all on the new standard-ACH path.

- Root cause: the payment sub-type arrived empty, so the pricing lookup matched nothing and no fee was written, with no error raised. The empty value came from two drifting versions of one SQL function.
- Pushback: there was pressure to exclude ACH from the check. I argued to keep it, since the fees were really missing.
- Fix: I wrote the RCA, including why our tests missed it. After the stopgap, I removed the duplicate-function design entirely, so this class of drift has no path back.

The rule I pushed: a payment type the system can't price must fail loudly.
Refund fees for large Venmo merchants and non-aggregated Amex.

- Root causes: I found two bugs in the backfill procedure (a time-window edge case and a filter that excluded pre-migration sales).
- Temu: I recovered ~$171K of refund fees across ~196K credits through the normal calculation path, with an audit trail.
- Reconciliation: I worked directly with the BT Account team and Temu's finance team, classified the 9,372 refund records they reported by cause, and mapped the recovered dollars to payouts.
- SHEIN: $33.9K backfilled and paid out.
- Non-aggregated Amex: I rebuilt the recovery path for refund fees, recovering 99 of 108 credits in the verified cohort.
- Runbook: I turned the procedure into a runbook with pre- and post-validation, so the next recovery is repeatable.

Incident ownership.

- ~$4M Amex payout delay for a large marketplace merchant. I was one of three named fix owners, and the senior engineer on the call deferred to me on the US Amex SLA. I set the line between expected delay and the real regression (our internal SLA is T+7, and we pay out only after Amex settles), and argued for a region-specific fix over a revert. Closed in three days.
- Terraform release failure. I root-caused and fixed it the same day, then proactively flagged two more resources with the same flaw.
- My own change. When one of my refund-fee changes caused a calculation incident, I owned it end to end: revert, post-mortem, and a rebuild as two smaller, separately verified changes. My rule since: ship the narrowest gate first, and test the failure path first.

Dive Deep and Are Right, A Lot: data over opinion

- 12M-transaction interchange gap. A coverage check had failed on 25 of 45 days, and the easy call was to mute it. I dug in and separated threshold noise from a real event: on 2026-07-20, 12.04M US card transactions were missing their expected interchange fee. I then recalibrated the check by region and pricing model. It has run 49 times since September without a failure.
- Async fee enrichment. I showed that ~92% of fee records missing their acquirer already had the data at detection, and ~7% more arrived within ~10 minutes. That turned a plan for open-ended backfills into a proposed small, durable queue (ADR in review). When another team preferred a shorter lookback window, I answered it with data: 2 months misses 20.9% of records, 120 days misses 2.5%.
- AU refund fees. An escalation bounced across teams before it reached me. I traced it to a merchant configuration setting, not a code bug, which ended the debate and became the AU refund-charge project.

- Validated against production before switching the source. Before switching interchange (the largest pass-through card fee) to the acquirer's earlier settlement feed, I ran the production query over a full day: 13.77M vs 13.74M rows, within 0.22%.

Invent and Simplify: platforms others build on

- Data-quality framework. I replaced one-off validation jobs with one config-driven framework:Each check is a config row.One runner isolates failures.Downstream jobs read a dataset only once all its checks pass."Is this data safe to read?" became a SQL condition instead of a Slack message, and adding a check became a config change instead of a deploy. I rolled it out to five datasets across US, EU and AU in about five weeks. Three other engineers have since added 13 checks of their own, and Express Lane consumes its readiness signal. I followed up with a design so one failing merchant never blocks the other ~1,240.
- Interchange source switch. For Net Settlement's daily payouts, I designed a fixed, per-fee-type switch to the earlier feed behind a toggle, rather than fragile runtime fallback logic, so the source of every fee is known in advance and auditable.
- MVPs and prototypes.DataLynk: I took over the data-contract ticket and merged the MVP in four weeks, writing 17 of its 21 commits.EU scheme-fee estimation engine: I co-built the rule engine (18 commits in its initial draft) and verified the new version produced identical outputs; the 194-rule reference set is in production.Prototypes: a prototype's 0.7% false-negative finding set the design of the coverage check that shipped.
- Simplifying what exists.Merged two drifting versions of a core SQL function into one.Moved a heavy event table to a lighter incremental design, sized for a much smaller warehouse.Replaced a check that failed every day with a per-region coverage check.Moved the noisiest check off a write window, cutting failed runs from 7.0 to 0.23 a week.

Think Big and Deliver Results: the next revenue lines

- US Gross Settlement fee (new Pricing product).Wrote the ADR (daily interest accrual on pass-through fees) and agreed the formula with Pricing Product, closing the first of its five decision items.Validated the timing on 13 production IC+ fee days, and ran a 10.4M-row production simulation for one account and cycle ($19K fee) cross-checked against a per-receipt calculation.Broke the program into 18 stories across Pricing, Funding, the event pipeline, Salesforce and QA, with a critical path to first charge.Built the implementation with integration tests, shipping switched off for the pilot.Walked Funding and Treasury through the design.Launch is set by Product's pilot-cohort decision, targeted for January 2027.
- AU refund charge (~AUD 100K a year at list price, 199 merchants). I found charges configured for AU merchants that Snowglobe was not collecting. My ADR weighed five options and caught that a naive build would overcharge merchants 10%, because stored prices include GST. I designed the correction and built the implementation, now in final review; launch follows Tax's ruling on the GST treatment.
- EU expansion. I extended the Amex settlement pipeline I built in 2025 to the EU, covering discovery, checklist, activation plan and code. Switched on 2026-04-28.
- Net Settlement ramp. I closed 16 stories supporting a 2026 ramp that brought 1,671 US merchants onto net fees in May, plus 11 AU merchants and Booking.

Earn Trust: influence beyond my code

- Domain reference. Partner teams route fee questions to me by name. A partner team lead redirected a review because "that table comes out of the … pipeline Chi built", and Funding's lead replied "Incredible! Thanks, Chi! We'll set this up for a test run today."
- Judgment under ambiguity. As interim owner of an intern-built ML anomaly detector that had only run on 6 test merchants (production has ~18K), I chose a go/no-go assessment over momentum. I froze alerting and CI/CD until the gate, and split the rollout into a rules-only phase and a gated ML phase.
- Knowledge that scales. 40 Confluence pages of designs, RCAs and ADRs. 74 PRs reviewed across 7 repos.
- AI adoption across the team.I shared my Claude Code skills toolkit with teammates and moved the team off Cortex Coco onto Claude Code, which materially cut our Cortex spend.Built ChiBot, a Claude-based Slack agent that answers routine data, PR and Jira questions when I'm away.Built a command that summarizes review status across repos.

Develop the Best: raising the people and the bar around me

- Mentorship.I mentor a new-grad engineer.I onboarded our summer intern. Before their last day I made sure the irreplaceable analyst labels were exported and wrote the handoff and cutover plan. I now own the detector's six-ticket roadmap.
- Practices I set for the team.The framework's "read only after checks pass" contract.Validating with the production query itself before a source switch.The post-mortem rule I took from my own incident: ship the narrowest gate first.
- Teaching through artifacts. Runbooks, an authoring guide for new checks, and a backfill template, so teammates can do this work without me.

Goals status

- Business impact and operational rigor: Strong: ~$205K recovered, two major fee gaps closed, on-call noise cut ~97%
- 30+ technical documents: Exceeded: 40
- Revenue-line delivery: Gross Settlement and AU designed and built, in final review; launch timing set by Product and Tax
- ADRs (goal 12): 4 substantial ADRs authored, each backed by production analysis; open for review
- EU scheme-fee engine: 194-rule reference set and helpers in production; accuracy measured once estimate generation runs
- AI adoption: Team moved to Claude Code with my skills toolkit, cutting Cortex spend; 2 automations built (ChiBot, review-status command)
- Office Hours and forum talk: Cross-team design walkthroughs this year; Office Hours and a larger forum committed for 2027
- Code reviews (goal 100+): 74 PRs across 7 repos
- Event Based Billing, Billing App Migration, CA: Deprioritized in favor of fee correctness and the new revenue lines

Growth

This year I grew from solving the problem in front of me to setting direction:

- writing the ADR before the code
- decomposing a cross-team program into a critical path
- deciding when not to ship (the anomaly detector)
- turning my own incident into a team rule

Challenges

Both new revenue lines are designed and built, and in final review. Gross Settlement is waiting on Product's pilot-cohort decision (targeted January 2027), and the AU charge is waiting on Tax's GST ruling. Next year I will name a decision owner and a decision date for each cross-team dependency in the ADR itself, so launch timing is visible from day one.

Support I'd value

- Help getting Product and Tax decisions on Gross Settlement and the AU charge scheduled early in Q1.
- A standing ADR review slot.
- A speaking slot at Pricing Office Hours.

2027 commitments

- Launch the Gross Settlement fee with the pilot cohort and own it in production.
- Launch the AU refund charge, merchant-level readiness and async enrichment.
- Measure the EU estimation engine's accuracy against actual fees.
- Report before/after numbers for three AI automations.
- Present at Pricing Office Hours and one larger forum, and grow the engineer I mentor into an independent owner.

Appendix: supporting detail

Reference material for readers who want the evidence behind the summary. The summary above stands on its own.

A. The year in numbers

- PRs authored: 58 across 4 repos: 40 merged, 4 in final review, 14 closed. Excludes 2 opened by a Copilot agent.
- Snowglobe rank: 36 merged PRs, 13th of 84 contributors by merged PRs, 11th by lines merged
- Breadth: 287 files across 24 modules: 47 stored procedures and prod-op scripts, 20 SQL functions, 12 scheduled tasks, 57 migrations, 51 test paths
- Jira: 71 tickets, 163 story points; 101 points done across 41 tickets (median 3); 16 discovery tickets (31 points); 82% of sprint-tracked tickets closed in sprint; 5 Jira projects
- Confluence: 40 pages (35 in 2026): 12 design docs, 9 discoveries, 6 RCAs, ADRs, 2 post-mortems, plus runbooks, how-tos and templates
- Code review: 74 PRs reviewed across 7 repos, 70 approved (counted by review date within the cycle)
- Slack: 210+ of my own posts in the two main Snowglobe engineering channels; dozens of threads answering Post-processing, BT Account, ML, Data and Funding

The arc

- Nov–Jan: build the platform.
- Feb–Apr: scale it and harden production.

- May–Jul: own the incidents.
- Aug–Oct: clean up, then design the next layer.

B. Where the work landed

- Refund fees: 16 · 10 · 4.3K · Recovered fees owed for Temu, SHEIN and non-agg Amex; fixed the backfill procedure
- Quality-check framework: 10 · 10 · 4.0K · The data-quality platform teammates build on
- Pass-through / interchange: 6 · 5 · 2.3K · Earlier source for interchange; per-region coverage check; retired a failing legacy check
- Health checks & alerting: 6 · 4 · 2.0K · On-call noise down; real gaps surfaced
- American Express: 5 · 3 · 2.5K · EU expansion, reject handling, OptBlue enablement
- ADRs: 2 · — · 0.9K · Gross Settlement fee, async enrichment
- Infrastructure: 3 · 2 · 1.0K · Reporting share, ownership-grant fix
- Gross Settlement: 1 · — · 1.5K · Next Pricing revenue line
- Acquirer enrichment, SQL hygiene, balances, reporting: 9 · 6 · 2.0K · Structural cleanup and the DataLynk MVP

- Platform / architecture: 17% · 36% · Fewer, larger changes that others build on
- Feature: 25% · 37% · New capabilities
- Incident follow-up: 13% · 6% · Root-cause fixes
- Operational work: 45% · 21% · Backfills, prod-ops, data fixes, alert tuning: the work that keeps money correct

C. Eight platform changes with lasting leverage

- Quality-check framework (#1997). Config-driven checks, failure isolation and a readiness handshake. Five dataset rollouts followed within about five weeks; teammates have since added 13 checks.
- DataLynk MVP (snowflake-reporting #22, Terraform #814). A new data path from Snowglobe into regional reporting schemas for disputes, fees, transfers and transactions. 17 of its 21 commits are mine.
- Interchange source switch (#1615, #1939). Moves the largest pass-through fee to an earlier feed. A fixed per-fee-type split; validated on a full production day (13.77M vs 13.74M rows); behind a toggle.
- Pass-through coverage check (#3040). Expected coverage per region, country and pricing model. It replaced a merchant-level check that failed every day.
- EU Amex prerequisites (#2598). Extended the Amex file pipeline to the EU aggregator flow without touching the US path.
- US Gross Settlement fee (#3785, ADR 1664). A new fee stream: interest-rate loader, accrual and posting, with integration tests, shipping switched off.
- Async fee enrichment (ADR 1528). A durable queue of incomplete records with a 5-minute worker, sized on large-sample production data.
- SQL-function consolidation (#3390, #3490). One canonical function, with the legacy signature dropped by a versioned migration, so the duplicate path behind the ACH gap is gone.

D. Eight operational fixes that kept production correct

- Temu refund-fee backfill (#2246). ~190K refunds run through the normal calculation path with an audit trail.

- Pre-migration backfill fix (#2689). Refunds on sales from before the platform migration were excluded; now captured.
- Window-boundary fix (#2763). Credits that straddled the backfill window were dropped; fixed, with tests.
- Stuck-fee cleanup (#2753). Cleared fees stuck behind a calculation race.
- Pre-migration credits (#2966). Each credit confirmed as a full refund against source data before insert.
- Volume-based paging (#3361). The non-agg Amex check now pages on unmatched volume, not on any merchant being present.
- Schedule move (#3408). The same check moved off the refund-fee write window, taking failed runs from 7.0 to 0.23 a week.
- Pinless false errors (#3187). Stopped false validation errors for payment types with no configured fee (~31M junk rows in a July sample).

E. Exploration that shaped what shipped

Several prototype and spike PRs were closed after their findings moved into the final design:

- Transaction-level interchange check: 0.7% false negatives on production US data · Set the design of the shipped coverage check (#3040)
- Transfer-time acquirer backfill: ~82% of gaps from an ingest race, ~18% from chargeback lookback · Root-cause section of ADR 1528
- Async enrichment implementation: 598-second recovery bound, 92/7/0.4% split · Sizing of ADR 1528
- Alert threshold dry run: 90-day replay: 18,505 → 1,046 alerts · Retune shipped in #3361

The rest were duplicates or drafts recreated cleanly (#2716 → #2717, #2965 → #2966, #3226 → #3237).

F. Technical depth: how the key systems work

American Express pipeline (built 2025, extended to the EU this year). Amex sends one fixed-width reconciliation file a day, so its transactions wait in a pending state until the file confirms settlement. The pipeline runs in five steps:

- Load the file into staging.
- Split it into six record streams, each with its own offset.
- Merge each stream into a typed table on a composite key.
- Validate the file against pending transactions with a full outer join, so every mismatch is logged, never dropped.
- Price the confirmed transactions in a task gated on both upstream steps.

It is idempotent at three layers (file dedup, merge keys, record-don't-drop validation) and has a single kill switch.

Quality-check framework.

- A contract table holds one row per (dataset, validation): procedure, date column, SLA and parameters.
- A generic executor runs each check dynamically and normalizes the result to PASS, FAIL or WARNING. An exception handler turns any crash into a FAIL row instead of a stopped run.
- When every enabled check for a dataset passes, a status row flips to Completed and is replicated to the share layer. Consumers join on it, so readiness is enforced in SQL.
- The merchant-level design keeps the table-level block as the strictest rule, so mixed deployments are always safe.

Interchange source switch.

- The merge key is synthetic: prefix plus invoice number, so hourly reruns never double-charge.
- A one-month time window on the merge condition keeps scans bounded as the table grows.
- A readiness join ensures a settlement day is used only after its checks pass.
- Old and new tasks are mutually exclusive under one toggle, so interchange comes from exactly one source at a time.

ACH silent-fee gap.

- Signal: a health check, "every settled transaction has its Braintree fee", flagged the gap.
- Isolation: slicing showed 100% of misses were bank-account payments on the new standard-ACH path. A control group with the field populated had fees, and coverage was zero at every age, which ruled out lag.
- Mechanism: the pricing category is built by concatenating a prefix with the payment sub-type. With the sub-type empty, the concatenation produced nothing, so no category matched.
- Root cause: the shared merge used a version of the function with no bank-account branch, overwriting a value already computed correctly.
- Structural fix: one function with a defaulted second parameter, plus a versioned migration that drops the old signature.

Schema pool (personal developer tooling). Each development session borrows a zero-copy clone of a schema kept in sync with the commit deployed to production. Each clone takes about two seconds, because only metadata is copied. That lets several sessions, including AI coding agents, run migrations and integration tests in parallel without colliding.

G. Design documents

- AU Refund Charge Rollout: ADR · Five options compared; GST-inclusive pricing would overcharge 10% without a ×10/11 correction
- Gross Settlement Fee (ADR 1664): ADR · Daily accrual posted before the IC+ fee day; validated on production data
- Missing Acquirers (ADR 1528): ADR · Durable enrichment queue sized on production data
- Merchant-Level Quality Checks: ADR · Sparse failure rows; 96.4% first-pass rate means near-zero cost
- Standard ACH fee gap: RCA · Keep the alert; fix the function drift
- Non-agg Amex refund recovery: Post-mortem · Shipped and verified; 99/108 recovered in the verified cohort
- Temu Venmo refund fees: Discovery · Sized the gap; drove the backfill
- Non-agg threshold capture failure: RCA · A timing race; backtest went from 9/10 failing runs to 0/10 after the move
- EU NET/DAILY balances: RCA · False positive, not missing money; 20/20 clean on replay
- Datahub → Snowglobe CDC sync: RCA · 14,282 stale rows back to 2024; cross-team finding
- Health-check task failures + monitor: Discovery and tool · Traced failures; 97.9% of affected refunds recoverable
- Fee Anomaly Detector handoff: Plan · State, handoff and production go/no-go
- Event table migration: Design · Dynamic table to stream-and-task, sized for a much smaller warehouse
- Venmo refund ProdOps: Runbook · Repeatable recovery with pre- and post-validation

- Row-count threshold guide: How-to · Authoring guide for new checks, backed by 35 integration tests
- EU Amex readiness and activation: Checklist and plan · Pre-rollout checks, activation plan, production verification

H. Cross-team reach

- Funding: EU Amex test run; refund-fee design review; Gross Settlement walkthrough
- Treasury: Gross Settlement design walkthrough
- Pricing Product: Gross Settlement formula agreed
- Post-processing: Acquirer-enrichment design discussion, answered with data (~10 threads)
- BT Account and merchant finance: Temu and SHEIN reconciliation (~20 exchanges)
- ML: Interchange data validation for their models
- Data engineering: Unblocked local migration testing; trigger-status contract walkthrough
- Partner pricing team: Reviews redirected to me as the pipeline's author

I. PR references

- Framework: #1997, #2061, #2064, #2069, #2075, #2091, #2106, #2175, #2185, #2558
- Interchange: #1615, #1939
- Amex: #1988, #2598, #2717
- Refund fees: #2246, #2689, #2753, #2763, #2966, #3141, #3283, #3580
- Coverage and alerting: #3040, #3237, #3361, #3396, #3408, #3419, #3187
- SQL hygiene: #3390, #3490
- Gross Settlement: ADR 1664, #3785
- Enrichment: ADR 1528
- DataLynk: snowflake-reporting #22; snowglobe-terraform #814
- Infrastructure: snowglobe-terraform #1326

Question

Identify development areas for 2027, focusing on the skills and capabilities to develop for professional growth and future business impact.

Employee

Comment

2027 Development Areas — Chi Zhang

My 2026 work showed I can find what is broken in system, fix it at the root, and design what comes next. To operate like a Senior, I need to turn that individual depth into outcomes that ship through other people and other teams. These six areas are where I will grow, each tied to the team's 2027 priorities and each measured by results, not activity.

1. Driving cross-team decisions to a launch date

Where I am. Gross Settlement and the AU refund charge are designed and built, but their launch depends on decisions from Product and Tax. I finished my side; I did not yet shape the decision timeline.

What I will build. The habit of treating a dependency on another team as part of my deliverable:

- Every ADR names a decision owner and a decision date.
- I bring open questions to the right forum early, with options and a recommendation, not just a status.
- I escalate on time instead of waiting.

Measure.

- Gross Settlement reaches first charge with the pilot cohort in Q1.
- The AU refund charge launches in H1.
- My next two cross-team programs each have every decision owner and date agreed before implementation starts.

2. From well-argued designs to approved, adopted standards

Where I am. My 2026 ADRs are grounded in production data, but none has been approved, and I authored far fewer than my goal.
What I will build.

- Write shorter, decision-first ADRs that reviewers can approve in one sitting.
- Seek early buy-in from the owning teams before formal review.
- Review others' designs on a steady cadence, so I learn how the broader org decides and earn the same attention in return.

Measure.

- My open ADRs (Gross Settlement, async enrichment, merchant-level readiness, AU refund) reach a decision.
- One new ADR a month in the areas the team is investing in: global expansion, net settlement payment types, data products.
- Two or more design reviews for other engineers each month.

3. Growing other engineers into owners

Where I am. I mentor a new-grad engineer and supported a summer intern, but I can't yet point to a milestone they reached because of me. At Senior level, my impact should show up in what others deliver.
What I will build.

- A structured mentoring practice: a written growth plan, bi-weekly check-ins, and deliberately handing over work I would normally keep. Fee-correctness RCAs, health-check ownership and on-call shadowing are the obvious candidates.
- More of my know-how written down as reusable guides, so teammates can do the work without me.

Measure.

- My mentee independently ships one production feature and leads one RCA.
- At least one area I own today, such as the quality framework or the refund-fee recovery runbook, is co-owned by a teammate by year end.

4. Thinking in business outcomes, not only correctness

Where I am. My work protects revenue, but I usually describe it in rows and transactions, not in dollars and merchant experience. Gross Settlement's revenue is still unquantified.
What I will build.

- Start each initiative from the business question: what it earns, what it protects, which merchants feel it.
- Partner more closely with Pricing Product and Finance, so success metrics are agreed upfront and reported at completion.
- Use the EU estimation engine as the test case: measure its accuracy against actual fees and turn that into cost predictions Pricing can act on.

Measure.

- Each major 2027 launch has a dollar or merchant-impact target set before work starts and reported after.
- The EU estimation engine produces estimates with measured accuracy, and generalizing it to AU and CA is scoped.

5. Measured, team-wide leverage from AI

Where I am. I moved the team onto Claude Code, shared my skills toolkit and cut our Cortex spend. I also built ChiBot and other automations. What I haven't done is measure the gains, or make them the team's default way of working.
What I will build. AI adoption run like an engineering program:

- Pick three recurring workflows (fee-variance analysis, incident triage, data-quality reporting).
- Automate them.
- Publish before/after numbers and a playbook teammates can follow.
- Set guardrails for agents that touch production data.

Measure.

- Three automations in team use, each with a published time or quality gain.
- A team playbook for AI-assisted development and RCA.
- Adoption beyond my own work.

6. Being visible beyond my team

Where I am. My influence is strong within Pricing and its partner teams, but I gave no broader talks this year.
What I will build. The ability to explain complex systems simply to wider audiences, and to make Snowglobe's correctness patterns reusable across the org: readiness handshakes, validating against production before a source switch, and loud failure for unpriced payments.
Measure.

- Two Pricing Office Hours sessions.
- One larger forum talk.
- At least one partner team adopting a pattern I presented.

What success looks like by end of 2027

- Two new revenue lines live and measured.
- My designs approved and used as team standards.
- An engineer I mentor owning production work on their own.
- AI gains the team can quantify.
- I am recognized beyond Pricing as the go-to engineer for fee correctness.